BFS Code:

```
from collections import deque
def bfs_shortest_path(graph, start):
    visited = [False] * len(graph)
    distance = [-1] * len(graph)
    queue = deque()
    visited[start] = True
    distance[start] = 0
    queue.append(start)
    while queue:
        current = queue.popleft()
        for i in range(len(graph)):
            if graph[current][i] == 1 and not visited[i]:
                visited[i] = True
                distance[i] = distance[current] + 1
                queue.append(i)
    return distance
n = int(input("Enter number of nodes: "))
print("Enter adjacency matrix:")
graph = []
for _ in range(n):
    graph.append(list(map(int, input().split())))
start = int(input("Enter starting node: "))
result = bfs_shortest_path(graph, start)
print("Shortest distances from node", start)
for i in range(n):
    print(f"Node {i} : {result[i]}")
```

output:

AI.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
                distance[i] = distance[current] + 1
                queue.append(i)

    return distance


# -------- Main Program --------
n = int(input("Enter number of nodes: "))

print("Enter adjacency matrix:")
graph = []
for _ in range(n):
    graph.append(list(map(int, input().split())))

start = int(input("Enter starting node: "))

result = bfs_shortest_path(graph, start)

print("Shortest distances from node", start)
for i in range(n):
    print(f"Node {i} : {result[i]}")
```

```
Enter number of nodes: 4
Enter adjacency matrix:
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
Enter starting node: 0
Shortest distances from node 0
Node 0 : 0
Node 1 : 1
Node 2 : 1
Node 3 : 2
```

Variables Terminal

2023005263

Manasa.Y